Università degli Studi di Modena e Reggio Emilia

Facoltà di Ingegneria – Reggio Emilia

CORSO DI

RETI DI CALCOLATORI
Linguaggio Java:
Remote Method Invocation

Prof. Franco Zambonelli

A.A. 2005-2006

LA PROGRAMMAZIONE RMI

Programmare clienti e servitori con le Socket è completamente diverso dal programmare clienti e servitori secondo la programmazione ad oggetti normale:

Con gli oggetti

```
Counter c = new Counter();
…
c.inc();
```

modo per richiedere un servizio all'oggetto contatore,

Con le socket, il cliente deve fare:

```
Socket s = new Socket("pippo.com", 34);
// get input stream….etc..
//  invia I dati
//  ricevi le risposte…
s.close()
```

E il servizio del server:

```
ss = s.accept();
// get input e output stream
//ricevi I dati
//svolgi il servizio
//invia le risposte
//ss.close();
```

Sono metodi completamente diversi!!!

REMOTE METHOD INVOCATION (RMI)

L'idea è quella di poter chiedere a servizi ad oggetti sulla rete *COME SE FOSSERO LOCALI*

In pratica, l'idea è quella di creare la possibilità di avere accesso agli oggetti che sono su nodi diversi della rete Internet, e poter avere riferimenti a questi oggetti, per poi poter chidere loro servizi (cioè la esecuzione dei loro metodi) come se fossero oggetti normali.

```
OggettoOvunque cc;

cc.metodo();
```

Si usano gli strumenti di **RMI**

Permettono di usare oggetti "remoti" (cioè su altri nodi della rete Internet) come se fossero oggetti come tutti gli altri, senza doversi preoccupare della realizzazione dei protocolli, della connessione, degli stream di input e output, etc. etc.

Gli unici problemi aggiuntivi sono:

- il cliente deve in qualche modo trovare un riferimento all'oggetto remoto (è diverso dal creare l'oggetto o dal farsi dare un riferimento passato come parametro)
- l'oggetto che vuole rendere i suoi servizi invocabili da rete deve esplicitamente dire che vuole fare ciò e deve rendersi reperibile

RMI e il Concetto di "Middleware"

Potremmo considerare RMI come un ulteriore "livello" nella pila ISO-OSI, che si pone tra il livello di trasporto e il livello di servizi.

Tali servizi RMI includono:

- la possibilità per un oggetto di rendersi disponibile (con alcuni dei suoi metodi) a essere invocato da remote
- la possibilità di recuperare riferimenti ad oggetti remoti
- la possibilità di invocare metodi su oggetti remoti come se fossero locali

Spesso, si parla di questi servizi (che non sono proprio coerenti con il modello "service access point" dello stack ISO/OSI) come "servizi di middleware", qualcosa che si pone "in the middle" tra:

- OSware e
- SoftWare

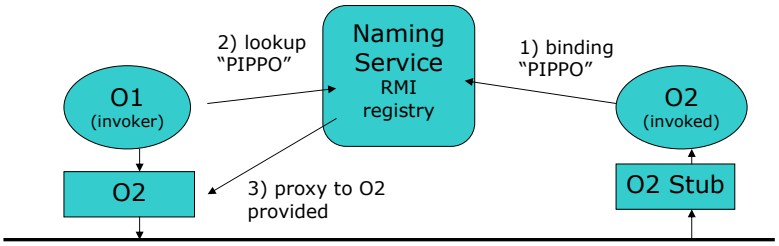


ARCHITETTURA RMI (1)

Architettura generale adottata anche da sistemi di middleware diversi (p.e., CORBA o Microsoft .NET)

Esiste un programma speciale che agisce da "Naming Service", come una specie di "yellow pages" → in Java è il "RMI Registry"

- registra i nomi e i riferimenti degli oggetti i cui metodi possono essere invocati da remoto (O2 nella figura)
- tali oggetti devono registrarsi, fare il "bind" presso il naming service, con un nome pubblico, "PIPPO" nella figura
- altri oggetti possono richiedere, fare il "lookup" di oggetti registrati chiedendo, a partire dal nome pubblico, un riferimento all'oggetto



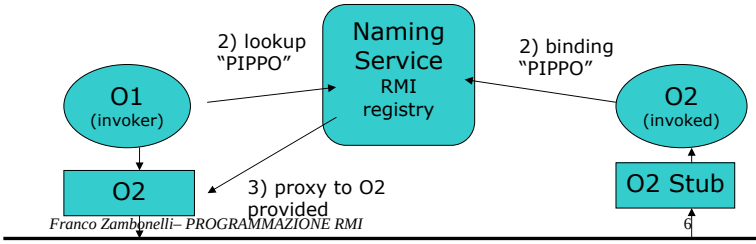
ARCHITETTURA RMI (2)

Quando il Naming Service riceve una richiesta di lookup, corrispondente a un oggetto che lui conosce (nella figura, il Naming Service sa che l'oggetto di nome pubblico "PIPPO" corrisponde all'oggetto O2, allora:

- viene generato un oggetto "virtuale", denominato "proxy" o anche "skeleton", che viene inviato presso il nodo di colui che aveva richiesto il lookup (O1 in figura)
- dal punto di vista di O1, esso percepisce il proxy di O2 come un normale riferimento a un oggetto O2 locale

In verità, quando O1 chiede un servizio a O2, lo chiede al suo proxy che:

- senza che O1 se ne accorga, stabilisce una connessione Socket TCP con il nodo dove c'e' O2 vero
- in tale nodo di O2, un componente associato a O2, detto "stub" riceve la connessione e la richiesta di servizio, e provvede a invocarla lui su O2 vero
- quando O2 vero risponde, lo stub manda la risposta indietro al proxy, il quale poi risponde a O1 come se fosse stato realmente lui a eseguire il servizio



DEFINIZIONE METODI RMI

Per creare un metodo/servizio RMI è necessario che sulla macchina server si operino i seguenti passi:

1. Definizione di un'interfaccia che specifichi i metodi/servizi invocabili da remoto; tale interfaccia deve ereditare dall'interfaccia *Remote.*
2. Avviare il registry RMI, che permette di pubblicare i metodi/servizi di questo tipo; si lancia <*start rmiregistry*> da linea di comando. RMI Registry agisce come una specie di "pagine gialle" della rete.
3. Definizione della classe che implementa i metodi/servizi vero e proprio; tale classe deve implementare l'interfaccia precedente (cioè definirne il codice dei metodi) ed ereditare dalla classe *UnicastRemoteObject*. Nel suo codice deve anche registrare (si dice più precisamente fare il "binding") il metodi/servizio presso l'RMI registry: in pratica deve "pubblicare" sulle pagine gialle il fatto che il metodo/servizio esiste ed è disponibile. Si dovrà stabilire e decidere un "nome" da assegnare (bind) al servizio.
4. Compilare la classe del punto precedente anche con il compilatore <*rmic nome_classe*>. Questo genera altre due file .class (nome_classe_Stub .class e nome_classe_Skel.class) che si occupano dei dettagli di basso livello della comunicazione client/server → realizzano lo stub e il proxy rispettivamente
5. Avviare l'applicazione che istanzia la classe server creata e registra i metodi/servizi nell'RMI registry

USO SERVIZI RMI

Le applicazioni client che vogliono usufruire del servizio di invocazione remota devono:

1. Conoscere il "nome" con cui è stato pubblicato sul registry il servizio che si richiede
2. Richiedere al registry di ottenere un riferimento all'oggetto che interessa → operazione di "lookup"
3. Tale interfaccia è quella definita in precedenza ed elenca i metodi invocabili

Per concretizzare i concetti e le fasi implementative fin qui descritte si presentano alcuni esempi.

ESEMPIO INTRODUTTIVO:Counter RMI

Definiamo l'interfaccia che tale contatore dovrà realizzare in questo modo:

```
import java.rmi.*;

public interface ContatoreInterface extends Remote {
public int getVal()  throws RemoteException;
public void inc()    throws RemoteException;
// notiamo l'eccezione che si puo' generare
}
```

E quindi definiamo il nostro contatore, facendogli implementare l'interfaccia di cui sopra e facendogli ereditare da UnicastRemoteException:

```
import java.rmi.* ;
import java.rmi.server.* ;
import java.rmi.registry.*;
import java.net.* ;
// tutte queste import sono necessarie

public class ContatoreServer
       extends UnicastRemoteObject
                    implements ContatoreInterface {
int valore;

public ContatoreServer() throws RemoteException
                           {valore =0;}
// notiamo l'eccezione che si puo' generare
// in fase di costruzione

public int getVal() {return valore;}
public void inc() {valore++;}
}
```


Counter RMI (2)

Compilare il contatore:

```
c:> javac ContatoreServer.java
c:> rmic ContatoreServer
```

Lanciare RMIRegistry

```
C:> rmiregistry
```


Counter RMI (3)

Il main che crea il contatore e ne pubblica il nome:

```
import java.rmi.* ;
import java.rmi.server.* ;
import java.rmi.registry.*;
import java.net.* ;
// soliti import

public class AvvioServer  {

public static void main (String[] args) throws
RemoteException, AlreadyBoundException,
MalformedURLException {
// notare le eccezioni da gestire


System.setSecurityManager(
             new RMISecurityManager());
// creiamo l'oggetto security manager
// faremo sempre cosi' per dare
// accesso a tutti indiscriminatamente

     ContatoreServer c = new ContatoreServer();
// creiamo il contatore c

     Naming.bind("155.185.200.60/mioconta", c);
// assegniamo il nome al nostro contatore c
// cioé facciamo il "binding"
// in questo caso, il nostro oggetto e' al
// nodo con indirizzo IP 155.185.200.60
// ed ha come nome "mioconta"
}
}
```


Counter RMI (4)

Ecco un client, da compilarsi e eseguirsi normalmente

```
import java.rmi.* ;
import java.rmi.server.* ;
import java.rmi.registry.*;
import java.net.* ;

public class ContatoreClient  {

public static void main (String[] args) throws Exception
{
// notiamo che invece di specificare tutti
// i tipi di eccezione, possiamo dire
// genericamente "Exception"

System.setSecurityManager(
            new RMISecurityManager());
// sistemiamo al solito modo le politiche
// di sicurezza

ContatoreInterface cc = (ContatoreInterface)
        Naming.lookup("155.185.200.60/mioconta");
// facciamo l'operazione di lookup
// e recuperiamo un riferimento all'oggetto
// attraverso la interfaccia
// dobbiamo sapere in che nodo è e come si chiama!!!

// e quindi usiamo l'oggetto come se
// fosse un normalissimo contatore
cc.inc();
System.out.println("Contatore " + cc.getVal());
}
}
```

UNA NOTA SULLA SICUREZZA IN RMI

Java RMI non è necessariamente sicuro. Infatti, io potrei trasferire tramite parametri su invocazione RMI degli oggetti da un computer all'altro

- ***dynamic class loading***: un oggetto trasferito via RMI potrebbe non avere il codice della classe sul computer sul quale viene traferito.In questo caso, Java provvede a trasferire ("loading") dinamicamente il codice della classe da un computer all'altro
- questo introduce problemi di sicurezza in quanto le nuove classi potrebbero fare azioni deleterie

Vedremo in seguito come in Java sia possibile gestire politiche di sicurezza (tramite il SecurityManager) che dicono cosa il codice delle classi (cioè gli oggetti di certe classi) possono e non possono fare.

Nei nostri esempi, per ora abbiamo dato accesso a tutti indiscriminatamente. Però, su alcuni sistemi Windows la gestione della sicurezza nel modo minimale che abbiamo fatto vedere potrebbe non funzionare. Se ciò succede, memorizzate nella cartella che contiene i vostri programmi un semplice file di nome policy con il seguente contenuto:

```
grant
{  // Allow everything for now
 permission java.security.AllPermission;
};
```

e quindi lanciate i vostri programmi (sia la parte server che la parte client) con il seguente comando:

```
java -Djava.security.policy=policy NomeClasse
```


I°) Esempio: Un server matematico

Supponiamo di avere a disposizione una macchina molto performante, accessibile in rete da altre postazioni meno potenti. Si può pensare di far svolgere a questa macchina delle funzioni di calcolo matematico e di rendere accessibili tali funzioni alle postazioni meno potenti. Possiamo realizzare il tutto scrivendo un'applicazione da far girare sul server e un'applicazione da far girare sui client.

Se scriviamo le applicazioni in Java possiamo sfruttare la possibilità di invocazione remota di metodi (RMI)

Creazione dell'interfaccia del server RMI:

```
import java.rmi.*;
public interface MathServerInterface  extends Remote{

float moltiplica (float x, float y)
    throws RemoteException;
float somma (float x, float y)
    throws RemoteException;
}
```


Implementazione dell'interfaccia del server RMI:

```
import java.rmi.*  import java.rmi.server.* ;
import java.rmi.registry.*; import java.net.* ;


public class MathServer  extends UnicastRemoteObject
implements MathServerInterface{


public float somma(float x,float y){return (x+y);}
public float moltiplica(float x,float y){return (x*y);}


public MathServer() throws RemoteException{}


/* il main è nella stessa classe!!! */
public static void main (String[] args) throws Exception
{


System.setSecurityManager( new RMISecurityManager());


MathServer server = new MathServer();
Naming.bind("PC_name/MathServer",server);
System.out.println("Server matematico pronto");
}
```

- l'RMISecurityManager è necessario ai controlli sulla sicurezza degli accessi alla macchina
- il nome del servizio che si è installato sul server è ***PC_name/MathServer***, dove PC_name è il nome di rete della macchina server

I°)ESEMPIO RMI: lato Server

Inoltre per rendere il servizio completamente attivo è necessario eseguire altre due operazioni:

- compilare la classe MathServer con il comando <*rmic*> (dopo averla compilata con *javac*). Questo genera i file *MathServer_Stub.class* e *MathServer_Skel.class*
- prima di eseguire l'applicazione avviare il registro dei servizi RMI con il comando <*start rmiregistry*>.

Questi ultimi passaggi, anche se possono apparire macchinosi, consentono di delegare al meccanismo dell'RMI una grande mole di dettagli implementativi, altrimenti difficilmente gestibili dal programmatore

I°)ESEMPIO RMI: lato Client

I client devono poter utilizzare il servizio reso disponibile dal server; per fare questo è necessario e sufficiente che essi conoscano il nome del servizio stesso e l'interfaccia che esso implementa: nel nostro caso rispettivamente ***PC_name/MathServer*** e ***MathServerInterface***.

```
import java.rmi.* ; import java.rmi.registry.*;

public class UsaMathServer{
public static void main(String[] args)
    throws Exception{
System.setSecurityManager(
    New RMISecurityManager());

//recupero del servizio remoto
MathServerInterface mathServer =
    (MathServerInterface)Naming.lookup(
    "PC_name/MathServer");

//uso del server matematico
// è come se fosse un oggetto locale !!!
float f = mathServer.somma(12.3, 16.96);
//…
}
```


II°)ESEMPIO RMI: Server di calcolo general-purpose (1)

Nell'esempio precedente il server metteva a disposizione la propria potenza di calcolo per l'esecuzione di alcuni metodi specifici: il codice di tali metodi era specificato dal lato del server. Sfruttando la tecnologia OO di Java è possibile implementare *un server di calcolo general-purpose, in grado di eseguire un qualsiasi "lavoro" (task), richiestogli dai client*. Il grande vantaggio di questo è non dover specificare dal lato server il codice dei metodi, ma permettere ai client di fornire l'intero task, con il relativo codice di esecuzione.

Per realizzare questo è necessario definire anche l'interfaccia che devono implementare i task da eseguire:

```
import java.io.* ;

public interface Task extends Serializable{
Object execute();
}
```

Quindi, in generale, un task è eseguito dal proprio metodo execute() e restituisce un Object. Un task è serializzabile perché deve essere passato dal client al server

II°)ESEMPIO RMI: Server di calcolo general-purpose (2)

Anche l'interfaccia remota del servizio RMI acquisisce generalità rispetto all'esempio precedente, e compendia un unico metodo generico:

```
import java.rmi.* ;

public interface Compute extends Remote{
Object executeTask(Task t)
    throws RemoteException;
}
```


II°)ESEMPIO RMI: Server di calcolo general-purpose (3)

La *classe server del servizio RMI* ha struttura analoga, ma delega l'esecuzione del task al task stesso:

```
import java.rmi.* ;
import java.rmi.server.* ;

public class ComputeEngine extends UnicastRemoteObject
implements Compute{

public ComputeEngine() throws RemoteException{}
public Object executeTask(Task t){
    return t.execute();
}

public static void main(Sting[] args)
    throws Exception{
System.setSecurityManager(
    new RMISecurityManager());
ComputeEngine engine = new ComputeEngine();
Naming.bind("PC_name/Compute", engine);
System.out.println("Compute Engine pronto");
}
```

Dopo aver compilato questi file con *<javac>* e quest'ultima classe anche con *<rmic>*, si può far partire il registry RMI (*<start rmiregistry>*) ed avviare l'applicazione che crea e registr ail servizio (in pratica la classe ComputeEngine)

II°)ESEMPIO RMI: utilizzo del Server di calcolo general-purpose(1)

Supponiamo di avere un'operazione computazionalmente onerosa da eseguire, ad esempio il calcolo di una serie di coefficienti per lo studio di un fenomeno meteorologico. Dato che il server non può conoscere l'algoritmo per calcolare tali coefficienti, definiamo un task che incapsuli dati e algoritmo per la risoluzione di questo problema.

Definizione del task:

```
public class Meteo implements Task{
 //…….
 //…variabili
 //….
 public Meteo (float[] input){……}

 //calcolo del risultato
 public Object execute(){
 //……
 //……
 return risultato;
 }
}
```


II°)ESEMPIO RMI: utilizzo del Server di calcolo general-purpose(2)

L'applicazione client che chiede al server di eseguire i task è piuttosto semplice:

```
import java.rmi.* ;
public class UsaEngine{
 public static void main(String[] args)
    throws Exception{
//…acquisizione degli input
//..
System.setSecurityManager(
    new RMISecurityManager());

//recupero del servizio remoto
Compute comp =(Compute)
    Naming.lookup("PC_name/Compute");

//preparazione di un task
Meteo task = new Meteo(input);

//esecuzione del task
Float[] res =
    (Float[])comp.executeTask(task);
//…}
```